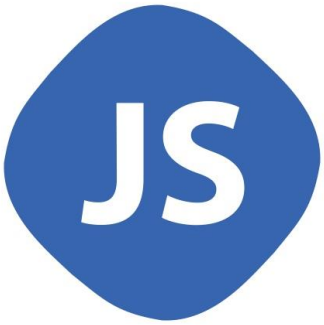


Подробнее о функциях

- Введение
- Переменные и константы в качестве функций
- Поднятие (hoisting)
- Строгий режим
- Функции и строгий режим
- Переопределение функций
- Вызов функции по месту (IIFE)
- Цепочка вызовов
- Функции в объектах (методы)

Введение

Перед ознакомлением с уроком необходимо повторить урок: **Блоковая область видимости**, темы: **Управляющие конструкции**.

Основные вопросы для повторения.

1. Области видимости (пространства имен):
    - **глобальная**;
    - **локальная**.
2. Глобальный объект **window**.
3. Типы локальных областей видимости:
    - **область видимости блока**;
    - **область видимости функции**;
    - **лексическая область видимости**.
4. **Объявленные** (явные) и **необъявленные** (неявные) переменные.
5. Область видимости, определяемая ключевыми словами:
    - **var**;
    - **let**;
    - **const**.



Переменные и константы в качестве функций

Подобно тому, как константам и переменным присваиваются простейшие значения (числа, строки и т.д.), также им можно **присваивать функции**, а точнее исходный код функции. Затем через такую переменную или константу можно вызвать присвоенную ей функцию.

> ***Примечание***. Некоторые вопросы рассматривались нами в предыдущих уроках. Но для наилучшего восприятия текущего урока рассмотри их более подробно.

При тестировании кода демонстрационного примера анализируйте выводы в консоль браузера.

example_1. Присвоение переменной ссылки на функцию

```
<script>
    // определение функции
    function fnHello (name){
        return "Hello, my dear " + name + "!";
    }
    // присвоение переменной hello ссылки на функцию fnHello
    let hello = fnHello;
    // смотрим, что в переменной hello
    console.log (hello);
    // присвоение переменной result результата вызова функции
    fnHello
    let result = fnHello("User");
    / смотрим, что в переменной result
```



```
    console.log(result);
</script>
```

Важно. Чрезвычайно важно понимать, чем отличаются присвоение функции со скобками и без. В первом случае **возвращаем результат вызова функции,** во втором передаем **ссылку на функцию** (возвращаем тело / исходный код функции).

На понимании этого процесса будем разбираться с **коллбэками** и **замыканиями**.

Поднятие (hoisting)

Поднятие (**hoisting**) представляет процесс **доступа** к переменным до их **определения**.

Важно. **Подъем** — это поведение JavaScript по умолчанию, при котором все объявления перемещаются в **начало текущей области** (в начало текущего скрипта или текущей функции).

Данная концепция связана с работой компилятора JavaScript. Компиляция кода происходит в **два прохода**:

- При первом проходе компилятор **получает все объявления переменных, все идентификаторы**. При этом никакой код не выполняется, методы не вызываются.
- При втором проходе, собственно, **происходит выполнение**. И даже если переменная определена после непосредственного использования, ошибки не возникнет, так как при первом проходе компилятору уже известны все переменные.

То есть как будто происходит поднятие кода с определением переменных и функций вверх до их непосредственного использования.

Переменные var

var-переменные, которые попадают под **hoisting**, по умолчанию получают значение **undefined**. Например, выполнение следующего кода вызовет ошибку:

```
// переменная var без объявления
console.log(val); // Uncaught ReferenceError: val is not defined
```

Но при добавлении строки инициализирующей переменную в консоль будет выведено значение **undefined**.

```
// переменная var с объявлением
console.log(val);  // undefined
var val = "Oops...";
```

При первом проходе компилятор узнает про существование переменной **val**. Она получает значение **undefined**. При втором проходе вызывается метод **console.log**(val).

Обратите внимание, JavaScript поднимает только объявления (**var** val), а не инициализации (**=** "Oops...").

example_2. Hoisting var-переменных

```
<script>
    // вывод переменной var
    console.log(val);  // undefined
    // определение переменной var
    var val = "Oops...";
</script>
```



Разберем следующее выражение:

```
// переменные b и c не определены
var a = b * c;
// определение переменных var
var b = 2;
var c = 3;
console.log(a); // NaN
```

Переменные **b** и **c** используются до определения. По умолчанию им присваиваются значения **undefined**. А если умножить undefined на undefined, то получим **Not a Number** (NaN).

Переменные let, const

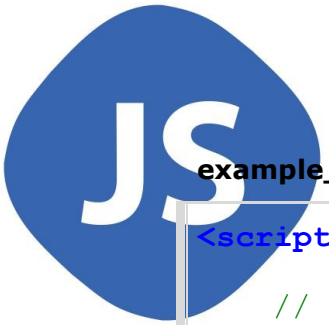
Процесс **hoisting** для let-переменных и констант будет отличаться: в отличие от var-переменных **константам** и **let-переменным** при первом проходе **не присваивается начальное значение**.

example_3. Hoisting let-переменных

```
<script>
    // Uncaught ReferenceError: Cannot access 'val' before
    initialization
    console.log(val);
    let val = "Hi!";
    console.log(val);
</script>
```

Функция инструкция

Все то же самое относится и к использованию функций. Мы можем сначала **вызвать** функцию, а потом уже ее **определить**.



example_4. Hoisting функций инструкций

```
<script>
    // вызов функции
    sayHello();
    // определение функции
    function sayHello(){
        console.log("Hello Hoisting!");
    }
</script>
```

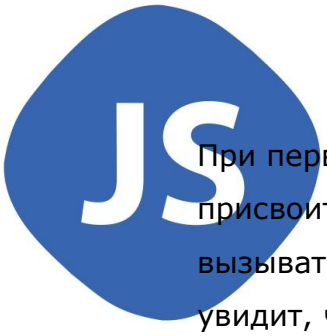
Функциональное выражение

По-другому ведут себя функции, когда они определяются в виде переменной – **функциональные выражения**.

example_5. Hoisting функций выражений

```
<script>
    // вызов функции
    // Uncaught TypeError: sayHello is not a function
    sayHello();
    // определение функции
    var sayHello = function () {
        console.log("Hello Hoisting!");
    }
</script>
```

В данном случае мы получим ошибку **Uncaught TypeError: sayHello is not a function**.



При первом проходе компилятор также получит переменную **sayHello** и присвоит ей значение **undefined**. При втором проходе, когда надо будет вызывать функцию, на которую будет ссылаться эта переменная, компилятор увидит, что вызывать нечего: переменная **sayHello** пока еще равна **undefined**. И будет выброшена ошибка.

Чтобы избежать возможных ошибок, всегда объявляйте все переменные в начале каждой области видимости. Это всегда хорошее правило, поскольку именно так JavaScript интерпретирует код.

> ***Важно***. JavaScript в строгом режиме не позволяет использовать переменные, если они не объявлены.

Строгий режим

Строгий режим (**strict mode**) — это важная часть современного JavaScript. Именно этот режим позволяет разработчикам пользоваться более **ограниченным**, чем стандартный, синтаксисом.

Семантика строгого режима отличается от традиционного нестрогого режима, который иногда называют **грязным** (**sloppy mode**). В таком режиме синтаксические правила языка не так строги, а когда происходят некоторые ошибки, система никак не оповещает о них пользователя.

> ***Важно***. Таким образом — **ошибки могут быть проигнорированы**, а код, в котором они допущены, сможет **выполняться дальше**. Это способно привести к неожиданным результатам выполнения кода.

Строгий режим вносит в семантику JavaScript некоторые изменения. Он не даёт системе закрывать глаза на ошибки, выдавая соответствующие исключения. Это приводит к **остановке выполнения программ**.



Строгий режим, кроме того, помогает в написании программ, в которых нет недочётов, мешающих JS-движкам оптимизировать код. В строгом режиме запрещено использование элементов синтаксиса, которые могут получить особый смысл в будущих версиях языка.

Особенности применения строгого режима

Особенности применения строгого режима:

- Можно применять к **отдельным функциям** или к **целому скрипту**.
- Нельзя применить только к **отдельным инструкциям** или к **блокам кода**, **заключённым в фигурные скобки**.

Для того чтобы использовать строгий режим на уровне целого скрипта, в самое начало файла, до любых других команд, нужно поместить конструкцию **"use strict"** или **'use strict'**.

Если в проекте имеются некоторые скрипты, в которых не используется строгий режим, и другие, в которых этот режим используется, тогда может случиться так, что эти скрипты окажутся объединены.

Это приведёт к тому, что код, который не предназначен для выполнения в строгом режиме, окажется в таком состоянии, когда система попытается выполнить его в строгом режиме. Возможно и обратное — код, написанный для строгого режима, попадёт в нестрогий режим. Поэтому лучше всего не смешивать строгие и нестрогие скрипты.

> ***Важно***. В JavaScript-модулях, которые появились в стандарте ES2015, строгий режим включён по умолчанию. Поэтому при работе с ними включать его явным образом не нужно. Модули будем изучать в следующих уроках текущей темы.



Функции и строгий режим

Для того чтобы применить строгий режим к отдельным функциям - конструкцию **"use strict"** или **'use strict'** надо поместить в верхнюю часть тела функции, до любых других команд.

Строгий режим при таком подходе применяется ко всему, что размещено в теле функции, включая **вложенные функции**.

```
var strictFunction = () => {
        'use strict';  // включаем строгий режим для функции
    const nestedFunction = () => {
        // эта функция тоже использует строгий режим
    }
}
```

Перед примерами использования функций несколько слов о глобальном объекте и глобальном контексте.

Глобальный объект

Глобальный объект — это корневой объект в программе. Если мы запускаем JS-код в браузере, то **глобальным объектом** будет **window**.

Ключевое слово this

Ключевое слово **this** — это ссылка на некий объект, к свойствам которого можно получить доступ. **This** называют **контекстом выполнения**. Контекст выполнения бывает:

- **Глобальный**.
- **Локальный**.

Глобальный контекст

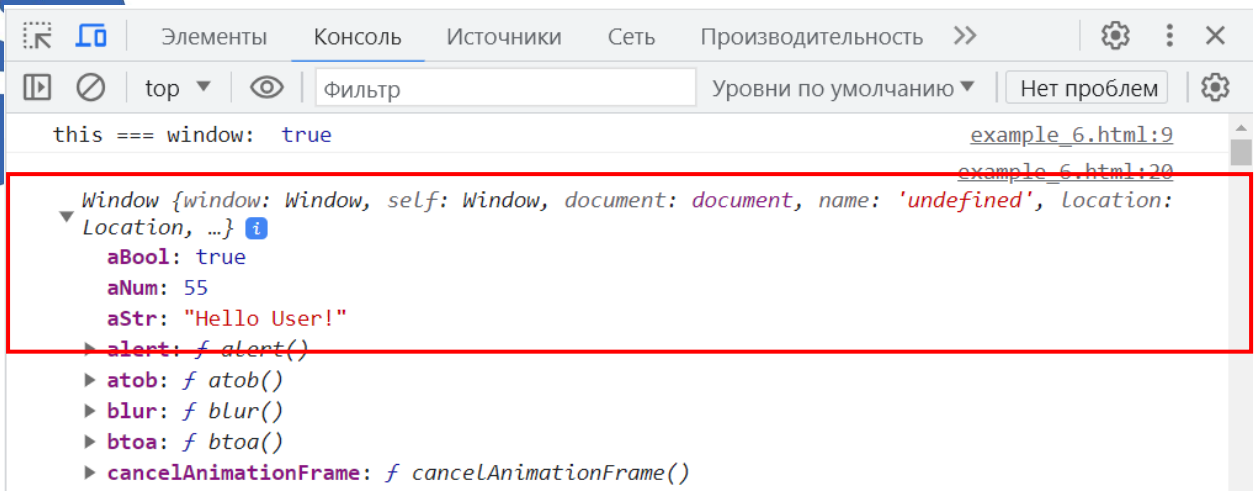


В глобальном контексте выполнения (за пределами каких-либо функций) **this** ссылается на **глобальный объект window** вне зависимости от режима (строгий или нестрогий).

example_6. Глобальный контекст

```
<script>
    // глобальный контекст
    console.log("this === window: ", this === window); // true
    // числовая переменная станет глобальным свойством объекта window
    aNum = 55;
    // строковая переменная станет глобальным свойством объекта window
    window.aStr = "Hello User!";
    // логическая переменная станет глобальным свойством объекта window
    this.aBool = true;
    console.log(this);
    // console.log(aNum);
    // console.log(window.aStr);
    // console.log(this.aBool);
</script>
```

Примечание. К именам глобальных переменных я добавляю префикс "**a**" только с одной целью – так их проще искать в списке свойств глобального **window**.



Функциональный контекст

В пределах функции значение **this** зависит от того, в каким режиме вызвана функция:

- грязный режим **sloppy mode** (по умолчанию) или
- строгий режим **strict mode**.

Грязный режим (sloppy mode)

Когда мы вызываем функцию, значением **this** может быть:

- **глобальный объект**, по умолчанию;
- **undefined**, при использовании конструкции **use strict**.

При отсутствии конструкции **use strict** скрипт автоматически используется в **нестрогом** (грязном) **режиме**.

example_7. Грязный режим

```
<script>

    // нестрогий режим / sloppy mode

    function fnSloppy() {

        // числовая переменная станет глобальным свойством
        объекта window
```

```
        aSloppy = 99;
        return this;
    }
    console.log("НЕстрогий режим ->");
    // this ссылается на глобальный window
    console.log(fnSloppy() === window); // true
    console.log(this); // глобальный объект window
    // глобальный window содержит свойство aSloppy
    console.log(this.aSloppy); // 99
</script>
```

Строгий режим (strict mode)

Чтобы включить **строгий режим** необходимо использовать конструкцию **use strict**.

example_8. Строгий режим

```
<script>
    // строгий режим / strict mode
    function fnStrict() {
        "use strict"; // включаем strict mode
        // пытаемся переменную сделать глобальным свойством
        объекта window
        aStrict = 99;
        return this;
    }
    console.log("Строгий режим ->");
    // this НЕ ссылается на глобальный window
```

```
    console.log(fnStrict() === window); // false
    // this ссылается на undefined
    console.log(fnStrict() === undefined); // true
    console.log(this);
</script>
```

Код демонстрационного примера завершился с ошибкой из-за желания создать глобальную переменную **aStrict** на объекте **undefined**. В демонстрационном примере **example_9** рассмотрим варианты восстановления работоспособности скрипта.

example_9. Правим код

```
<script>
    function fnStrict() {
        "use strict";
        // пытаемся переменную сделать глобальным свойством
        объекта window
        aStrict = 99;
        /*
        код далее выполняться не будет
        для выполнения необходимо:
        либо:
            закомментировать - use strict;
        либо:
            закомментировать - aStrict = 99;
        либо:
            объявить переменную - var aStrict = 99; || let
            aStrict = 99;
```

```
        */
        return this;
    }
    fnStrict();
    console.log(this);
</script>
```

Попробуем отловить ошибку программно.

example_10. Используем обработку ошибок

```
<script>
    function fnStrict() {
        "use strict";
        // поместим код в блок обработки ошибок
        try {
            // пытаемся переменную сделать глобальным свойством
            объекта window
            aStrict = 99;
            console.log(this);
        } catch(err){
            // вывод информации об ошибке
            console.log ("Ошибка (" + err.name + "): " +
            err.message);
        }
        return this;
    }
    fnStrict();
</script>
```

Примечание. Обработку ошибок смотрите в теме: **Управляющие конструкции**, урок: **Обработка ошибок**.

Переопределение функций

Функции обладают возможностью для **переопределения поведения**. Переопределение происходит с помощью присвоения анонимной функции переменной, которая называется так же, как и переопределяемая функция.

В демонстрационном примере **example_11** продемонстрировано переопределение функции **message**().

example_11. Переопределение исходного кода функции

```
<script>
    function message(){
        console.log("Здравствуйте!");
        // переписываем значение глобальной переменной message
        message = function(){
            console.log("До свидания!");
        }
    }
    // первый вызов функции
    /*
        во время первого вызова в теле функции произошло
        переопределение исходного кода вызываемой функции
    */
    message();
```

```
    // второй вызов функции (функция переопределена)
    message();
</script>
```

При первом вызове функции действует основной блок операторов функции, в данном случае:

1. выводится сообщение "Здравствуйте!";
2. происходит переопределение тела функции **message**.

Таким образом, при всех последующих вызовах функции срабатывает ее **переопределенная версия**, а на консоль будет выводиться сообщение "До свидания!".

При переопределении функции надо учитывать некоторые нюансы. В частности, попробуем присвоить ссылку на функцию переменной и через эту переменную вызвать функцию.

example_12. Переопределение через дополнительную переменную

```
<script>
    function message() {
        console.log("Здравствуйте!");
        message = function() {
            console.log("До свидания!");
        }
    }
    // присвоение переменной displayMessage исходного кода функции message
    var displayMessage = message;
    /*
        теперь переменная displayMessage содержит следующее
```

```
        значение:
        {
            console.log("Здравствуйте!");
            message = function() {
                console.log("До свидания!");
        }
    */
    // console.log(displayMessage);
    /*
        - вызываем функцию displayMessage()
        - при этом перепишем переменную message
    */
    displayMessage();
    message();
</script>
```

В следующем демонстрационном примере вывод в документ блоков **header** и **footer** осуществляется с помощью переопределения функции **outScreen**().

example_13. Вывод блоков header и footer с помощью переопределения

```
<script>
    function outScreen() {
        // первый вызов функции выведет в браузер строку
        document.write (`
            <header>
            <h1 align="center">Функции JavaScript</h1>
            </header>
```

```
        `);
        // при первом вызове функции поменяем ее исходный код
        outScreen = function(){
            document.write (`
                <footer align="center">
                <h3>Верстка | Дизайн | Программирование</h3>
                <a href="https://vk.com/pechora_pro"
                target="_blank">pechora_PRO</a>
                </footer>
            `);
        }
    } // end function outScreen()
</script>
<!-- /// -->
<script>
    // первый вызов функции
    outScreen();
</script>
<h2>Переопределение функций</h2>
<script>
    // второй вызов функции
    outScreen();
</script>
<!-- /// -->
```

Дополнительная практика переопределения функций в одной из самостоятельных работ.

Вызов функции по месту (IIFE)

Когда функция обернута в пару "( )", мы можем вызвать эту функцию, добавив еще одни "( )" в конце, например:

```
( function foo(){ .. } ) ();
```

> ***Важно***. Первая окружающая пара ( ) делает функцию **выражением**, а вторая () - **выполняет функцию**.

```
// сразу после объявления вызов на исполнение (вызов по месту)
(function foo() {
     alert("Hello my friend!"); // Hello my friend!
})();
```

Этот шаблон настолько в ходу, что существует термин для его обозначения: **IIFE** (**Immediately Invoked Function Expression** - **немедленно вызываемое функциональное выражение**).

IIFE не обязательно нужны имена, самая распространенная форма IIFE — использование анонимного функционального выражения. Несмотря на это, например, именование IIFE (как именование функции) имеет все вышеперечисленные преимущества над анонимными функциональными выражениями, поэтому взять его на вооружение — хорошая практика.

Функция IIFE легко работает с **внешними** данными любых типов.

```
var num = 2;
// вызов по месту
(function () {
    var a = num + 3;
    console.log(a); // 5
}) ();
```



В следующих демонстрационных примерах тестируем варианты функций определения и исполнения **по месту**.

example_14. Функция IIFE выводит данные, объявленные внутри себя

```
<script>
    // определение/вызов функции по месту
    ( function fnSetData(login, pwd, email) {
        // инициируем объект
        let data = {
            login : login,
            pwd : pwd,
            email: email
        };
        // вывод объекта в консоль браузера
        console.log("вызов функции по месту:");
        console.log(data);
    } ) ("Мастер", "12345", "master@mail.ru");
</script>
```

В предыдущем варианте функция использовала данные, объявленные внутри себя, теперь поработаем с данными, инициализированными во внешнем коде.

example_15. Функция IIFE получает на вход внешние данные

```
<script>
    // определение объекта
    let person = {
        surname: "Строганова",
        name: "Ксения",
```

```
        post: "Преподаватель",
        email: "strog-x@gmail.com",
        phone: "2-12-85-06",
        experience: 14.5,
        img: "ava.jpg"
    };
    // определение/вызов функции по месту
    ( function ouDataPerson (data) {
    // интерполируем свойства объекта в строку
    with (data){
        html = `
            <img src="${img}"> <br>
            Фамилия:  <b> ${surname} </b><br>
            Имя: <b> ${name} </b><br>
            Должность: <b> ${post} </b><br>
            E-mail: <b> ${email} </b><br>
            Телефон: <b> ${phone} </b><br>
            Стаж: <b> ${experience} </b><br>
        `;
    }
    // вывод html-строки в документ браузера
    document.write(html);
    } ) (person);
</script>
```

Другой вариант **синтаксиса** традиционной формы IIFE:

```
( function(){ .. } () );
```



Эти две формы **идентичны** по функциональности. Какую из них предпочесть — всего лишь ваш стилистический выбор.

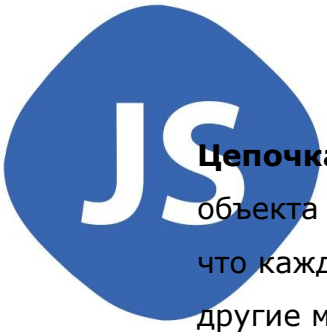
example_16. Еще один вариант синтаксиса функции IIFE

```
<script>
    // сохраняем коллекцию узлов h3
    let node = document.querySelectorAll("div h3");
    // определение/вызов функции по месту
    ( function getNode() {
        let couplets = new Array();
        // сохраняем текстовое содержимое узлов в массив
        for(let elem of node) {
            couplets.push(elem.innerText);
        }
        // вывод массива в консоль браузера
        console.log("вызов функции по месту:");
        console.log(couplets);
    } (node) );
</script>
```

Цепочка вызовов

Достаточно часто в кодинге мы использовали цепочку вызовов **стандартных** функций (методов).

```
// цепочка вызовов функций (методов)
let node = document.getElementById("01").querySelectorAll("h3");
```



Цепочка функций — это стиль программирования, при котором методы объекта (или функции) вызываются **последовательно**. Это возможно, потому что каждый метод возвращает объект, и следовательно, можно вызывать другие методы на этом объекте. Это обычно используется для создания интерфейса, который легко читается и понимается.

Рассмотрим, каким образом можно создать цепочку из вызовов **пользовательских** функций.

> ***Примечание***. Цепочка, построенная не на объектах, а на пользовательских функциях – это программный *костыль*. Но это работает, а объекты еще не изучали, поэтому пробуем.

Выполним последовательное создание интересующего нас кода функций.

example_17. Подготовка функций для организации цепочки

```
<script>
    // функция инициализации объекта данных
    function setData(login, pwd, email) {
        // в глобальный window запишем объект данных data
        data = {
            login : login,
            pwd : pwd,
            email: email
        };
    }
    // функция получения данных
    // получаемые данные зависят от входного параметра
    function getData (val) {
        // данные data берутся из глобального window
```

```
        if (val == "login") return data.login;
        if (val == "pwd") return data.pwd;
        if (val == "email") return data.email;
    }
    // вызов функции инициализации данных
    setData("Админ", "12345", "master@mail.ru");
    // получение данных email
    let email = getData("email");
    // вывод полученных данных в консоль браузера
    console.log(email);
</script>
```

Для организации цепочки, вызывающая функция должна вернуть объект **this**.

example_18. Цепочка вызовов пользовательских функций

```
<script>
    // функция инициализации объекта данных
    function setData(login, pwd, email) {
        // в глобальный window запишем объект данных data
        data = {
            login : login,
            pwd : pwd,
            email: email
        };
        return this;
    }
    // функция получения данных
```

```
    // получаемые данные зависят от входного параметра val
    function getData (val) {
        // данные data берутся из глобального window
        if (val == "login") return data.login;
        if (val == "pwd") return data.pwd;
        if (val == "email") return data.email;
    }
    // получение данных через цепочку вызовов
    let value = setData("Мастер", "12345",
    "master@mail.ru").getData("login");
    console.log(value);
</script>
```

Закрепим материал созданием нескольких функций, объединенных в цепочку, для работы с **внешним файлом данных**.

example_19. Цепочка вызовов для работы с внешними данными

```
<script src="teams-albums.js"></script>
<!-- ... -->
<script>
    // поиск данных в подключаемом массиве
    function fnSearchData(name) {
        // проходим по объектам массива
        for(let team of teams) {
            if (team.name == name) {
                // если текущий объект искомый -
                // - сохраняем объект в переменную teamData
                // - прерываем итерации
```

```
            teamData = team;
            break;
        }
    }
    return this;
}
// форматируем объект данных для вывода
function fnFormatData() {
    with(teamData) {
        teamOut = `
            <h3><a href='?id=${id}'>${name}</a></h3>
            Страна: <b>${country}</b><br>
            О группе: <b>${content}</b><br>
            Дата основания: <b>${date}</b><br>
            Стиль исполнения: <b>${style}</b><br>
            <a href='${link}'>Смотреть в Википедии</a><br>
        `;
    }
    teamOut += "<h3>Альбомы:</h3><ol>";
    // проходим по массиву альбомов
    for (team of teamData.albums) {
        teamOut += `<li>${team}</li>`;
    }
    teamOut += "</ol>";
    return this;
}
```

```
    // вставляем форматную строку в документ
    function fnOutData(target) {
        // выбор элемента по его id и вставка контента
        document.getElementById(target).innerHTML = teamOut;
    }
    // формируем цепочку вызовов
    fnSearchData("Pink
    Floyd").fnFormatData().fnOutData("console");
</script>
```

Функции в объектах (методы)

Мы очень много работаем с **объектами** и его **свойствами**. Объекты обычно создаются, чтобы представлять **сущности реального мира**, будь то пользователи, заказы и так далее:

```
// объект пользователя
let user = {
    name: "John Smith",
    login: "super-john",
    email: "super-john@mail.com"
};
// объект заказа
let order = {
    date: "12.06.23",
    bill: "09N-125289/2",
    vendor: "John Smith",
    products: [/* ... */]
```

```
};
```

И если в реальном мире пользователь может совершать **действия**: добавлять в корзину, авторизовываться, выходить из системы, оплачивать заказы и т.п, то и объект должен уметь выполнять такие же **действия**.

Действия объекта JavaScript представлены в виде специальных свойств, значениями которых выступают **функции**.

> ***Важно***. Функцию, которая является свойством объекта, называют **методом этого объекта**.

Ниже представлен объект пользователя со свойствами: **name**, **login**, **email** и методом **welcome**().

```
// объект пользователя
let user = {
    // свойства
    name: "John Smith",
    login: "super-john",
    email: "super-john@mail.com",
    // метод
    welcome: function () {
        alert ('Добрый день. Спасибо за визит!');
    }
}
// вызов метода
user.welcome();
```

Стиль программирования, в котором программист пишет код, используя объекты для представления сущностей реального мира, – называется **объектно-ориентированным программированием** или сокращённо: **ООП**.



Примечание. Объектно-ориентированное программирование является большой предметной областью и представляется отдельной темой курса.

В демонстрационном примере **example_20** создаем объект с тремя свойствами-функциями (методами) для поиска, форматирования и вывода в браузер объекта подключаемого массива.

example_20. Функции в качестве методов объекта

```
<script src="teams.js"></script>
<!-- ... -->
<script>
    // определение объекта со свойствами-функциями (методами)
    let obj = {
        // поиск данных в подключаемом массиве
        searchData: function(name) {
            // проходим по объектам массива
            for(let team of teams) {
                if (team.name == name) {
                    // если текущий объект искомый
                    return team;
                }
            }
        },
        // форматирование объекта данных для вывода
        formatData: function (data) {
            with(data) {
                formatTeam = `
```

```
                    <h3><a href='?id=${id}'>${name}</a></h3>
                    Страна: <b>${country}</b><br>
                    О группе: <b>${content}</b><br>
                    Дата основания: <b>${date}</b><br>
                    Стиль исполнения: <b>${style}</b><br>
                    <a href='${link}' target='_blank'>Смотреть в
                    Википедии</a><br>
                `;
            }
            return formatTeam;
        },
        // вставка форматной строки в документ
        outData: function (data) {
            // выбор элемента по его id и вставка контента
            document.getElementById("console").innerHTML = data;
        }
    }
    // последовательный вызов функций-методов
    // поиск
    let team = obj.searchData("Pink Floyd");
    // форматирование
    team = obj.formatData(team);
    // вставка
    obj.outData(team);
</script>
```



Закрепляем пройденный материал. В **самостоятельных работах** выполним следующие задания:

- Область видимости переменных var, let, const.
- Работа с глобальным объектом window.
- Ревьюирование кода функций, создающих цепочки вызовов.
- Написание переопределяющих функций.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы.

